

Plateforme d'Éducation Numérique — Bénin, Afrique de l'Ouest

25 Apps	Django 5.2	IA Multi-Provider	FedaPay XOF	WebSockets	Multi-Établissements
---------	------------	----------------------	-------------	------------	----------------------

TABLE DES MATIÈRES

1. Présentation du Projet	3
2. Objectifs & Périmètre	3
3. Acteurs & Rôles	4
4. Diagramme de Cas d'Utilisation	5
5. Architecture Technique	6
6. Stack Technologique Complet	7
7. Modélisation des Données	9
8. Fonctionnalités Détaillées	11
9. Plan d'Action — 8 Phases	14
10. Sécurité & Performance	17
11. Déploiement & Infrastructure	18
12. Critères de Qualité	19

1. PRÉSENTATION DU PROJET

1.1 Contexte

L'Académie Numérique est une plateforme éducative complète conçue spécifiquement pour le contexte béninois et africain. Elle répond au besoin de modernisation du système éducatif en offrant une solution tout-en-un : gestion des examens, correction par intelligence artificielle, bulletins numériques officiels, paiements via Mobile Money et communication en temps réel.

Champ	Valeur
Nom du projet	Académie Numérique
Domaine	EdTech — Éducation Numérique
Zone géographique	Bénin (Afrique de l'Ouest) — exportable Africa
Langue principale	Français (+ Anglais, Espagnol, Arabe)
Monnaie	Franc CFA — XOF (+ extension multi-devise possible)
Framework	Django 5.2 (Python 3)
Déploiement	Render.com (cloud, auto-scaling)
URL production	academie-composition.onrender.com
Modèle économique	SaaS — Abonnements FREE / PRO / ELITE + Frais scolaires

1.2 Vision

Devenir la plateforme de référence pour la gestion scolaire numérique en Afrique francophone, en combinant intelligence artificielle, temps réel et paiements mobiles adaptés au contexte local.

2. OBJECTIFS & PÉRIMÈTRE

2.1 Objectifs Fonctionnels

#	Objectif	Priorité	Statut
OF-01	Gestion complète des examens (création, planification, attribution)	■ Critique	■ Implémenté
OF-02	Compositions en ligne avec surveillance anti-triche	■ Critique	■ Implémenté
OF-03	Correction automatique par IA multi-provider (Groq/Gemini/NVIDIA)	■ Critique	■ Implémenté
OF-04	Génération de bulletins PDF officiels (format Ministère Bénin)	■ Critique	■ Implémenté
OF-05	QCM interactif avec banque de questions et notation avancée	■ Haute	■ Implémenté
OF-06	Examens nationaux (BEPC, CEP, Baccalauréat toutes séries)	■ Critique	■ Implémenté
OF-07	Paiements Mobile Money via Fedapay (MTN, Orange, Wave)	■ Haute	■ Implémenté
OF-08	Gamification : XP, badges, leaderboard, streaks	■ Normale	■ Implémenté
OF-09	Notifications temps réel (WebSockets, Django Channels)	■ Haute	■ Implémenté
OF-10	Certifications numériques vérifiables (token QR)	■ Normale	■ Implémenté
OF-11	Détection de plagiat entre copies	■ Normale	■ En cours
OF-12	Gestion multi-établissements (campus, classes, niveaux)	■ Haute	■ Implémenté

2.2 Objectifs Non-Fonctionnels

- **Performance** : Réponse < 200ms pour 95% des requêtes, support 500+ utilisateurs simultanés
- **Sécurité** : HTTPS obligatoire, CSRF, XSS protection, audit trail complet
- **Disponibilité** : 99.9% uptime (SLA Render), déploiement sans interruption
- **Accessibilité** : Responsive mobile-first, compatible réseau lent (Afrique)
- **Maintenabilité** : Code Django standard, tests unitaires, documentation inline
- **Internationalisation** : Français/Anglais/Arabe/Espagnol via Django i18n

3. ACTEURS & RÔLES

Rôle	Description	Permissions Clés	Accès
■■■■ Admin	Super-administrateur de la plateforme	• Gérer tous les utilisateurs • Valider examens nationaux • Config. établissements • Rapports financiers	is_staff + is_superuser
■ Conseiller Pédagogique	Coordinateur académique, superviseur des enseignants	• Approuver les épreuves • Voir tous les résultats • Gérer les classes • Tableau de bord complet	Role: conseiller
■■■■ Professeur	Enseignant créateur d'épreuves et correcteur	• Créer examens/devoirs • Uploader corrigés types • Valider corrections IA • Générer QCM	Role: professeur
■■■■ Élève	Apprenant, utilisateur final principal	• Passer examens/QCM • Voir bulletins • Télécharger certificats • Suivre XP et badges	Role: eleve
■■■■■■ Parent	Tuteur légal, accès lecture seule sur l'élève	• Voir bulletins enfant • Voir résultats • Payer frais scolaires • Recevoir notifications	App: parents

■■ Le modèle User utilise UUID comme clé primaire et génère automatiquement un matricule au format ROL-YYYY-00001 à la création.

4. DIAGRAMME DE CAS D'UTILISATION

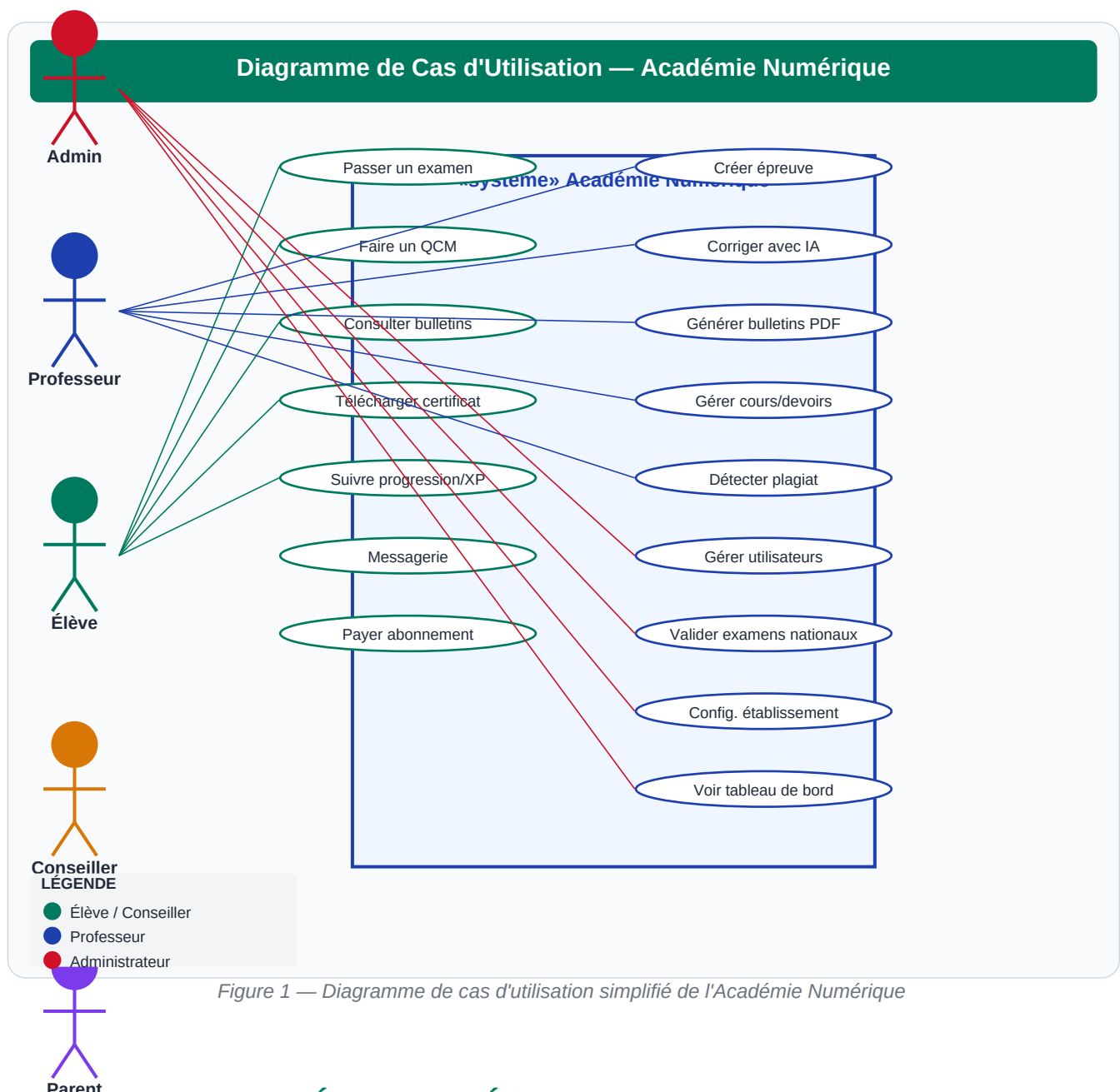


Figure 1 — Diagramme de cas d'utilisation simplifié de l'Académie Numérique

4.1 Cas d'Utilisation Étendus — Élève

ID	Cas d'Utilisation	Description
UC-E01	Passer un examen en ligne	Épreuve en mode plein écran, caméra activée, anti-triche temps réel
UC-E02	Soumettre copie papier	Upload photo de la copie → OCR → Correction IA
UC-E03	Participer à un QCM	Questions mélangées, timer, résultat immédiat ou différé
UC-E04	Consulter son bulletin	Accès bulletin PDF officiel avec signature numérique
UC-E05	Télécharger certificat	Certificat PDF avec QR code de vérification unique
UC-E06	Voir progression XP	Dashboard gamifié : niveau, badges, classement mondial/national

5. ARCHITECTURE TECHNIQUE

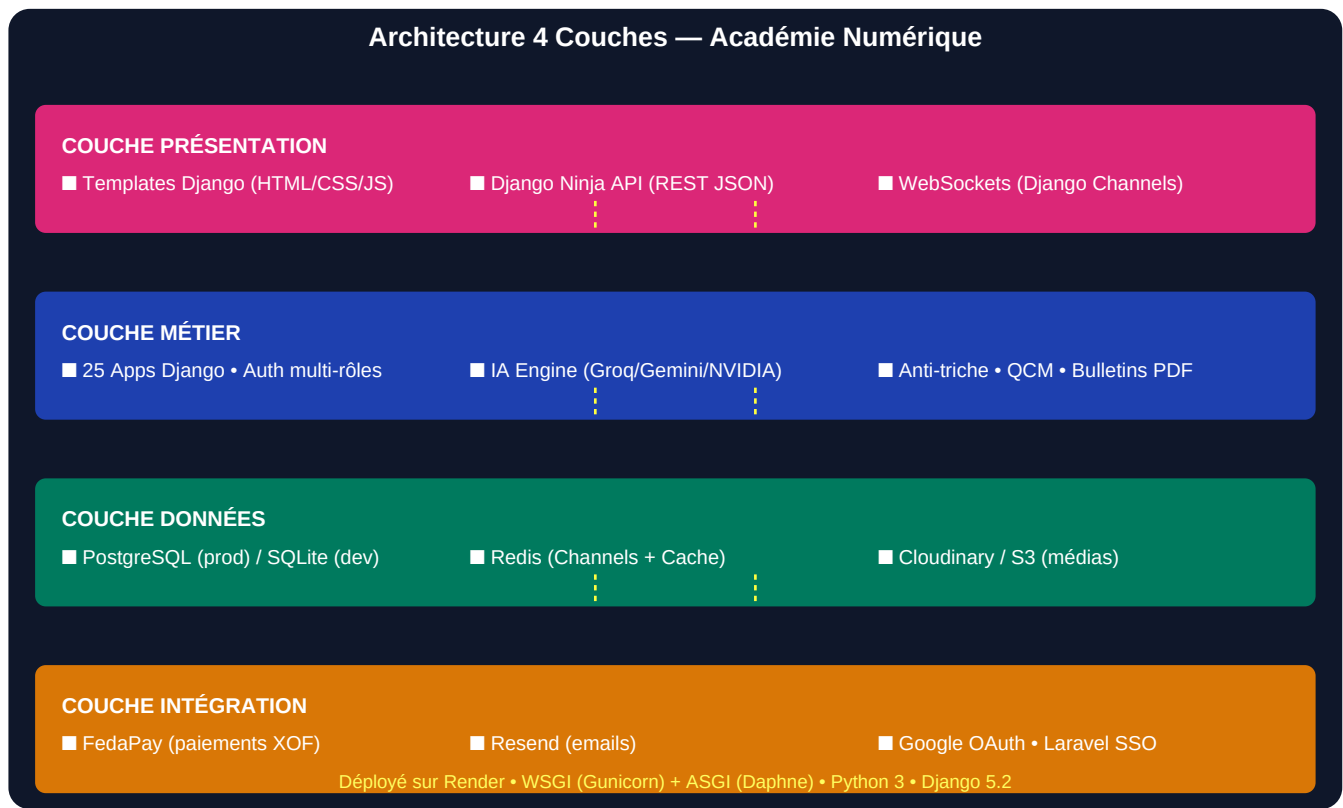


Figure 2 — Architecture 4 couches de l'Académie Numérique

5.1 Structure des Apps Django (25 applications)

App	Responsabilité	Modèles Principaux
core	Modèles et utilitaires partagés	Matiere, Classe, Feedback, CalendarEvent
accounts	Authentification & profils utilisateurs	User, Profile, Referral, Leaderboard
schools	Gestion multi-établissements	Etablissement, Campus, NiveauScolaire, Classe
exams	Épreuves et examens nationaux	Exam, ExamFile, ExamenNational, ExamenComposition
compositions	Sessions de composition en ligne/papier	CompositionSession, StudentAnswer, Resultat, AntiCheatLog
correction	Correction de copies uploadées	CorrectionCopie
ai_engine	Moteur IA multi-provider (Groq/Gemini/NVIDIA)	AICorrection, CorrectionDetail
bulletins	Génération bulletins officiels PDF	Bulletin, BulletinLigne
qcm	QCM interactif avec banque de questions	QuestionBank, QCMExam, QCMAnswer, QCMResultat

App	Responsabilité	Modèles Principaux
payments	Paielements scolaires et abonnements FedaPay	FraisScolaire, Paiement, AbonnementEleve, PaiementAbonnement
gamification	XP, badges, classement, streaks	Badge, UserBadge, GlobalLeaderboard, XPAction, StreakRecord
notifications	Notifications in-app et emails	Notification, EmailQueue, GlobalAnnouncement
certifications	Certificats numériques vérifiables	Certificate (QR + token)
plagiat	Détection de plagiat entre copies	PlagiatReport
realtime	WebSockets temps réel	Consumers Django Channels
audittrail	Journal d'audit complet	AuditLog
subscriptions	Plans d'abonnement plateforme	SubscriptionPlan, UserSubscription
cours / devoirs	Ressources pédagogiques	Cours, Devoir, Ressource
parents	Portail parents/tuteurs	Accès lecture bulletins élève
attendance / cahier	Présences et cahier de textes	Presence, CahierTexte
messaging	Messagerie interne	Message, Conversation
library	Bibliothèque numérique	Document, Resource
analytics	Tableaux de bord & statistiques	StatSnapshot, Report
webhooks	Réception webhooks externes (FedaPay)	WebhookEvent

6. STACK TECHNOLOGIQUE COMPLET

6.1 Backend

Technologie	Version	Rôle
Django	5.2.13	Framework web principal — ORM, templates, admin
Python	3.11+	Langage de programmation
Django Ninja	1.6.2	API REST moderne typée (OpenAPI auto-généré)
Django Channels	4.0.0	WebSockets et communications temps réel
Daphne	4.0.0	Serveur ASGI pour WebSockets
Gunicorn	25.3.0	Serveur WSGI pour requêtes HTTP classiques
Celery / Redis	redis 5.0.1	Queue tâches asynchrones (sans Celery — natif Redis)
WhiteNoise	6.12.0	Service fichiers statiques en production

6.2 Intelligence Artificielle

Provider	Modèle	Usage	Priorité
Groq	LLaMA 3 / Mixtral	Correction IA rapide, génération QCM	1 (primary)
Google Gemini	gemini-pro	Fallback correction, analyse pédagogique	2 (fallback)
Mistral AI	mistral-large	Correction multilingue	3 (fallback)
DeepSeek	deepseek-chat	Analyse mathématiques et sciences	4 (fallback)
NVIDIA NeMo	nemotron-4-340b	OCR avancé + correction haute précision	5 (premium)
Tesseract OCR	0.3.13	Reconnaissance texte copies papier (local)	Démo/local
PyMuPDF	1.27.2	Extraction texte PDF	Utilitaire

6.3 Base de Données & Stockage

Composant	Usage	Config
SQLite	Base de données développement local	DB_ENGINE=sqlite3
PostgreSQL	Base de données production (Render)	dj_database_url + DATABASE_URL

Composant	Usage	Config
MySQL	Option hébergement alternatif	DB_ENGINE=mysql, utf8mb4
Redis	Channels layer + Cache + Queue	REDIS_URL, channels-redis
Cloudinary	Stockage médias cloud (images, PDF)	USE_CLOUDINARY_STORAGE=true
AWS S3	Alternative stockage cloud (DigitalOcean Spaces)	USE_S3_STORAGE=true
Local FileSystem	Médias en développement	MEDIA_ROOT

6.4 Paiements & Intégrations Externes

Service	Usage	Détails
FedaPay	Paiements Mobile Money (Bénin)	MTN MoMo, Orange Money, Wave — XOF — Sandbox + Production
Resend	Envoi emails transactionnels	3000 emails/jour gratuit — SMTP api.resend.com
Google OAuth	Connexion Google (SSO)	GOOGLE_CLIENT_ID + GOOGLE_CLIENT_SECRET
Laravel SSO	Authentification inter-plateforme	LaravelAuthBackend custom — laravel_id/token sur User
Cloudinary API	Upload/transformation images	cloudinary://KEY:SECRET@CLOUD_NAME

7. MODÉLISATION DES DONNÉES

Modélisation — Diagramme Entité-Relation Principal

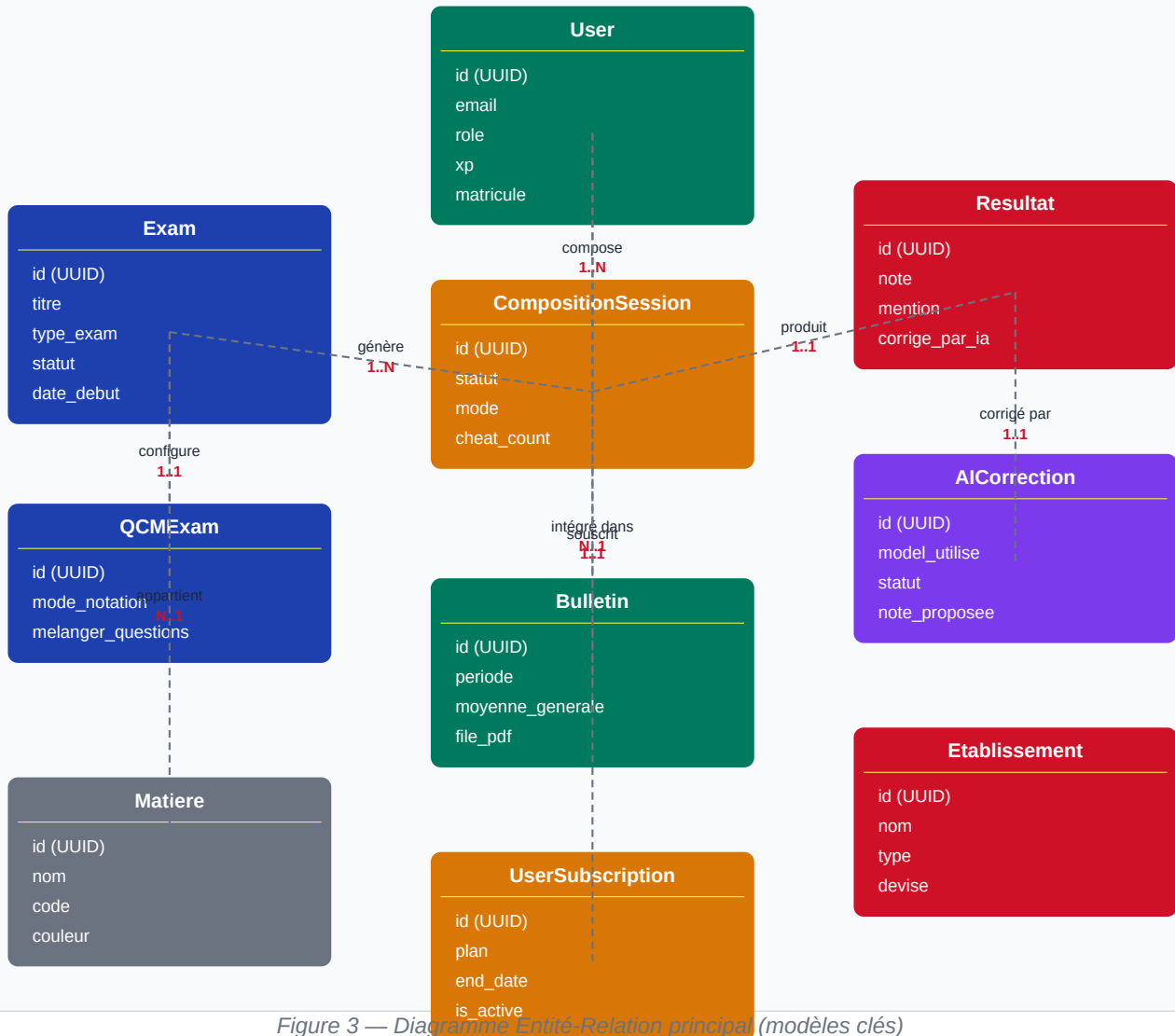


Figure 3 — Diagramme Entité-Relation principal (modèles clés)

7.1 Description des Entités Principales

7.1.x User (accounts)

Champ	Type / Contrainte
id	UUID (PK)
email	EmailField (unique)

Champ	Type / Contrainte
role	choices(admin conseiller professeur eleve)
niveau	choices(primaire secondaire universitaire ...)
xp	PositiveIntegerField
matricule	CharField (auto-généré ROL-YYYY-NNNNN)
laravel_id	FK SSO Laravel
dark_mode	BooleanField
preferred_language	CharField

7.1.x Exam (exams)

Champ	Type / Contrainte
id	UUID (PK)
titre	CharField(300)
type_exam	choices(composition examen devoir concours)
matiere	FK → core.Matiere
classe	FK → core.Classe
createur	FK → User
date_debut/fin	DateTimeField
statut	choices(brouillon publie en_cours termine)
anti_cheat_active	BooleanField
approval_status	choices(pending approved rejected)

7.1.x CompositionSession (compositions)

Champ	Type / Contrainte
id	UUID (PK)
exam	FK → Exam
eleve	FK → User
mode	choices(en_ligne sur_papier)
statut	choices(en_attente en_cours soumis corrige exclu)
cheat_count	PositiveIntegerField

Champ	Type / Contrainte
cheat_logs	JSONField
qr_code	ImageField
ip_address, user_agent	tracking

7.1.x Resultat (compositions)

Champ	Type / Contrainte
id	UUID (PK)
session	OneToOne → CompositionSession
note	DecimalField(5,2)
note_sur	DecimalField (défaut 20.00)
mention	choices(excellent tres_bien bien ...)
details_correction	JSONField
corrige_par_ia	BooleanField
corrige_par_humain	BooleanField
bulletin_pdf	FileField

7.1.x ExamenNational (exams)

Champ	Type / Contrainte
id	UUID (PK)
type_examen	choices(bepc cep bac_a1 bac_a2 bac_b bac_c bac_d ...)
annee_scolaire	CharField
coefficients_par_matiere	JSONField
classes	M2M → core.Classe
matieres	M2M → core.Matiere
statut	choices(brouillon programme_publie en_cours termine)

8. FONCTIONNALITÉS DÉTAILLÉES

8.1 Module Examens & Compositions

Création d'épreuve — Acteur : Professeur

- Formulaire de création avec titre, matière, classe, durée, dates
- Upload fichier épreuve (PDF) et corrigé type
- Activation optionnelle anti-triche (caméra, plein écran obligatoire)
- Mode de composition : en ligne ou sur papier
- Workflow d'approbation (pending → approved par conseiller/admin)

Composition en ligne — Acteur : Élève

- Interface plein écran obligatoire avec détection sortie
- Caméra surveillance (détection multi-visages)
- Suivi changements d'onglets (cheat_count)
- Timer countdown avec alerte 15min / 5min
- Sauvegarde automatique réponses (auto-save toutes les 30s)
- Soumission finale avec QR code de confirmation

Correction IA — Acteur : Système

- OCR de la copie papier (Tesseract/NVIDIA NeMo)
- Extraction texte corrigé type (PDF → text via PyMuPDF)
- Prompt IA structuré avec barème et critères
- Fallback automatique : Groq → Gemini → Mistral → DeepSeek → NVIDIA
- Note proposée + détail par question + points forts/erreurs
- Validation humaine obligatoire avant publication

8.2 Module Bulletins

Le système génère automatiquement des bulletins PDF au format officiel du Ministère de l'Éducation du Bénin. Un signal Django **post_save** sur le modèle Resultat déclenche la génération.

- Périodes : T1, T2, T3, S1, S2, Annuel, QCM
- Types : Administratif (Ministère), Professionnel, QCM
- Données incluses : notes par matière, coefficient, moyenne, rang, effectif, comportement, absences
- Appréciation pédagogique générée par IA (synthèse contextualisée)
- Signature numérique + token de vérification unique
- Export PDF avec WeasyPrint / ReportLab
- Envoi email automatique aux parents via Resend

8.3 Module QCM

- Banque de questions réutilisable par matière et difficulté (facile/moyen/difficile)
- Génération automatique de QCM par IA à partir d'un cours uploadé
- Modes de notation : classique, avec pénalité, partielle
- Mélange questions et choix (anti-triche)
- Résultat immédiat ou différé configurable
- Feedback IA détaillé par question avec explications
- Intégration au bulletin (QCMResultat → BulletinLigne)
- Contexte béninois : QCM spécifiques programmes Bénin/BEPC/BAC

8.4 Module Paiements (Fedapay)

- Frais scolaires : inscription, scolarité, examen, uniforme, transport, cantine...
- Paiement en espèces, Mobile Money (MTN/Orange/Wave), virement, carte
- Abonnements plateforme : Basique / Standard / Premium / Élite
- Échéanciers de paiement avec rappels automatiques (3j, 1j avant)
- Bourses scolaires (excellence, mérite, sociale, sportive)
- Rapports financiers PDF (mensuel, trimestriel, annuel)
- Codes promo avec réduction % ou montant fixe
- Configuration Fedapay par établissement (sandbox ↔ production)

8.5 Module Gamification

- Système XP : points gagnés à chaque action (examen, QCM, connexion...)
- Badges par catégorie : académique, participation, excellence, progression, social, spécial
- Rareté des badges : commun, rare, épique, légendaire
- Classement mondial et national avec périodes jour/semaine/mois/année/all-time
- Streaks : série de jours consécutifs d'activité
- Niveau calculé à partir des XP accumulés
- Système de parrainage (Referral) avec bonus XP

8.6 Module Temps Réel (WebSockets)

- Django Channels 4.0 + Redis Layer pour les connexions WebSocket
- Notifications push instantanées (nouvelle note, bulletin disponible, message)
- Tableau de bord temps réel des résultats d'examens
- Chat/messagerie en temps réel entre professeur et élèves
- Mise à jour live du leaderboard durant les examens
- Surveillance anti-triche en temps réel (alertes immédiates)

9. PLAN D'ACTION — 8 PHASES

Ce plan est séquentiel. Chaque phase doit être validée (tests + démo) avant de passer à la suivante. Les phases 1-3 constituent le MVP (Minimum Viable Product).

PHASE 1		FONDATIONS	■ 2 semaines
Environnement			
01	Créer le virtualenv Python et installer Django 5.2		■
02	Configurer le projet Django : academie_numerique/settings.py		■
03	Configurer .env avec toutes les variables (SECRET_KEY, DB, etc.)		■
04	Configurer la base de données PostgreSQL (prod) et SQLite (dev)		■
05	Initialiser Git, créer la structure de branches (main/develop)		■
Apps de base			
06	Créer l'app `core` : Matiere, Classe, Parametre		■
07	Créer l'app `accounts` : User custom (UUID, rôles, matricule auto)		■
08	Configurer AUTH_USER_MODEL = 'accounts.User'		■
09	Créer les migrations initiales et appliquer		■
10	Créer un superuser admin de test		■
Authentification			
11	Implémenter les vues login/logout/register		■
12	Configurer LaravelAuthBackend (SSO externe)		■
13	Implémenter Google OAuth (callback, token, création user)		■
14	Créer les templates auth (login.html, register.html)		■
15	Tester : connexion email/password + Google OAuth		■

PHASE 2		EXAMENS & COMPOSITIONS	■ 3 semaines
App exams			
01	Créer modèles : Exam, ExamFile, ExamAssignment		■
02	Créer modèles examens nationaux : ExamenNational, ExamenNationalMatiere		■
03	Vues CRUD examens (create/list/detail/delete)		■
04	Formulaires de création avec upload fichier PDF		■
05	Workflow approbation (pending → approved par conseiller)		■
06	Tester création et publication d'une épreuve		■
App compositions			
07	Créer modèles : CompositionSession, StudentAnswer, StudentSubmissionFile		■

08	Créer modèles : Resultat, AntiCheatLog	■
09	Vue 'salle d'examen' en ligne (plein écran, timer)	■
10	Implémentation anti-triche JS (tab change, fullscreen exit, copy-paste)	■
11	Système d'upload de copie papier (photo → serveur)	■
12	Signal post_save Resultat → trigger bulletin	■
13	Tester un cycle complet examen en ligne	■

PHASE 3**IA & CORRECTION****■ 3 semaines****App ai_engine**

01	Créer modèles : AICorrection, CorrectionDetail	■
02	Implémenter le service multi-AI avec fallback automatique	■
03	Intégrer Groq API (provider principal)	■
04	Intégrer Google Gemini (fallback 1)	■
05	Intégrer Mistral AI (fallback 2)	■
06	Intégrer NVIDIA NeMo (premium OCR)	■

OCR & Correction

07	Implémenter extraction texte PDF (PyMuPDF)	■
08	Implémenter OCR Tesseract pour copies papier	■
09	Implémenter OCR NVIDIA NeMo (haute précision)	■
10	Construire le prompt de correction (barème + critères)	■
11	Parser la réponse IA en JSON structuré	■
12	Vue validation humaine de la correction IA	■
13	Tester correction complète : upload copie → note IA → validation	■

PHASE 4**QCM & BULLETINS****■ 2 semaines****App qcm**

01	Créer modèles : QuestionBank, Choice, QCMExam, QCMAnswer, QCMResultat	■
02	Interface de création de questions (professeur)	■
03	Génération automatique de questions par IA (depuis cours)	■
04	Interface de passage du QCM (élève) avec timer	■
05	Calcul notes : classique, pénalité, partielle	■
06	Feedback IA post-QCM (explications détaillées)	■

App bulletins

07	Créer modèles : Bulletin, BulletinLigne	■
----	---	---

08	Service BulletinService.generate_pdf_from_bulletin()	■
09	Template PDF officiel (format Ministère Bénin)	■
10	Génération PDF avec WeasyPrint ou ReportLab	■
11	Signature numérique + token de vérification QR	■
12	Vue élève : télécharger son bulletin	■
13	Envoi email automatique (Resend)	■
14	Tester génération bulletin complet T1	■

PHASE 5**PAIEMENTS & ABONNEMENTS**

■ 2 semaines

FedaPay

01	Créer modèles payments : FraisScolaire, Paiement, EcheancierPaiement	■
02	Créer modèles abonnements : PlanAbonnementScolaire, AbonnementEleve, PaiementAbonnement	■
03	Créer ConfigurationFedaPay par établissement	■
04	Intégrer FedaPay SDK (initiation transaction)	■
05	Implémenter webhook FedaPay (confirmation paiement)	■
06	Pages paiement Mobile Money (MTN, Orange, Wave)	■
07	Génération reçu PDF post-paiement	■
08	Rappels automatiques d'échéances (3j, 1j)	■
09	Tester cycle complet paiement sandbox → production	■

PHASE 6**TEMPS RÉEL & NOTIFICATIONS**

■ 2 semaines

WebSockets

01	Configurer Django Channels avec Redis Layer	■
02	Configurer ASGI (asgi.py) + routage WebSocket	■
03	Créer consumers : NotificationConsumer, ExamConsumer	■
04	Implémenter connexion WebSocket côté client (JS)	■
05	Notifications push : nouvelle note, bulletin dispo, message reçu	■
06	Dashboard temps réel (résultats live pendant examens)	■

Notifications

07	Créer modèles : Notification, EmailQueue, GlobalAnnouncement	■
08	Service d'envoi email via Resend API	■
09	Notifications in-app (cloche + badge unread)	■
10	File d'attente emails (EmailQueue + worker Redis)	■

11 Tester : notification temps réel + email



GAMIFICATION & MODULES PHASE 7 ANNEXES ■ 2 semaines		
Gamification		
01	Créer modèles : Badge, UserBadge, GlobalLeaderboard, XPAction, StreakRecord	
02	Système d'attribution XP automatique (post-examen, QCM, connexion)	
03	Calcul et mise à jour du leaderboard mondial/national	
04	Attribution automatique de badges selon conditions	
05	Gestion des streaks (série jours consécutifs)	
06	Interface élève : mon niveau, mes badges, mon classement	
Modules complémentaires		
07	App cours : upload et gestion des cours par matière	
08	App devoirs : assignation et remise de devoirs	
09	App attendance : saisie des présences/absences	
10	App cahier : cahier de textes numérique	
11	App library : bibliothèque numérique	
12	App parents : portail accès lecture parents	
13	App certifications : certificats PDF avec QR code vérifiable	
14	App plagiat : détection similitude entre copies	

PHASE 8 DÉPLOIEMENT & OPTIMISATION ■ 1 semaine		
Déploiement Render		
01	Configurer render.yaml (web service + worker)	
02	Configurer PostgreSQL sur Render (DATABASE_URL)	
03	Configurer Redis sur Render (REDIS_URL)	
04	Configurer Cloudinary pour les médias (USE_CLOUDINARY_STORAGE=true)	
05	Variables d'environnement production (toutes les clés API)	
06	Configurer HTTPS / HSTS / Security Headers	
07	Configurer Daphne pour ASGI (WebSockets en prod)	
08	Tester le déploiement complet	
Optimisation & Sécurité		
09	Activer cache Redis (CACHE_BACKEND = django-redis)	
10	Optimiser requêtes ORM (select_related, prefetch_related)	

11	Configurer Content-Security-Policy headers	■
12	Audit trail : logger toutes les actions sensibles	■
13	Rate limiting sur API et authentification	■
14	Tests de charge (500+ utilisateurs simultanés)	■
15	Documentation API Swagger/OpenAPI (Django Ninja auto)	■
16	Configurer sauvegardes automatiques DB (Render backup)	■

10. SÉCURITÉ & PERFORMANCE

10.1 Mesures de Sécurité Implémentées

Mesure	Implémentation	Statut
CSRF Protection	CsrfViewMiddleware actif sur toutes les vues POST	■ Actif
XSS Protection	SECURE_BROWSER_XSS_FILTER + Content-Security-Policy header	■ Actif
Clickjacking	X_FRAME_OPTIONS = 'DENY'	■ Actif
HTTPS / HSTS	SECURE_SSL_REDIRECT + HSTS 1 an en production	■ Prod
Permissions-Policy	Désactivation caméra/micro/geo non autorisés	■ Actif
Session sécurisée	SESSION_COOKIE_SECURE + HTTPONLY + 7 jours	■ Actif
Mot de passe	Validators Django : longueur min 8, communs, numériques	■ Actif
Audit Trail	App audittrail : log toutes actions sensibles	■ Actif
Anti-triche	Tab change, fullscreen exit, multi-face, copy-paste detection	■ Actif
Rate Limiting	À implémenter sur login et API endpoints	■ Planifié
2FA	À implémenter (TOTP via django-otp)	■ Planifié

10.2 Variables d'Environnement Requises

```
DJANGO_SECRET_KEY=... # Clé secrète Django (50+ chars)
DEBUG=False # TOUJOURS False en production
DATABASE_URL=postgres://... # URL PostgreSQL
Render REDIS_URL=redis://... # URL Redis
Render GROQ_API_KEY=gsk_... # Provider IA principal
GEMINI_API_KEY=... # Fallback IA 1
MISTRAL_API_KEY=... # Fallback IA 2
NVIDIA_API_KEY=... # OCR premium
RESEND_API_KEY=re_... # Emails transactionnels
GOOGLE_CLIENT_ID=... # OAuth Google
GOOGLE_CLIENT_SECRET=... # OAuth Google
USE_CLOUDINARY_STORAGE=true # Stockage médias cloud
CLOUDINARY_URL=cloudinary://KEY:SECRET@CLOUD
SITE_URL=https://academie-composition.onrender.com
```

11. DÉPLOIEMENT & INFRASTRUCTURE

11.1 Configuration Render (render.yaml)

```
services: - type: web name: academie-numerique env: python buildCommand: pip install -r requirements.txt && python manage.py collectstatic startCommand: daphne -b 0.0.0.0 -p $PORT academie_numerique.asgi:application envVars: [...toutes les variables ci-dessus...] - type: worker name: academie-worker startCommand: python manage.py process_tasks # ou daphne worker databases: - name: academie-db databaseName: academie_numerique plan: starter - name: academie-redis plan: starter
```

11.2 Procfile (backup)

```
web: daphne -b 0.0.0.0 -p $PORT academie_numerique.asgi:application worker: python manage.py process_tasks
```

Étape	Commande
1. Installer dépendances	<code>pip install -r requirements.txt</code>
2. Variables d'env	<code>cp .env.example .env && éditer</code>
3. Migrations DB	<code>python manage.py migrate</code>
4. Collecter statiques	<code>python manage.py collectstatic --noinput</code>
5. Créer superuser	<code>python manage.py createsuperuser</code>
6. Lancer dev	<code>python manage.py runserver 0.0.0.0:8000</code>
7. Lancer prod (ASGI)	<code>daphne -b 0.0.0.0 -p 8000 academie_numerique.asgi:application</code>

12. CRITÈRES DE QUALITÉ & VALIDATION

12.1 Critères de Validation par Phase

Phase	Critère de Validation	Test
Phase 1	Un utilisateur peut s'inscrire, se connecter et accéder à son dashboard	Manuel
Phase 2	Un élève peut passer un examen en ligne de bout en bout	Manuel
Phase 3	Une copie uploadée est corrigée par l'IA avec une note valide	Manuel
Phase 4	Un bulletin PDF est généré et téléchargeable après un examen	Manuel
Phase 5	Un paiement Mobile Money FedaPay sandbox se complète correctement	Sandbox
Phase 6	Une notification temps réel est reçue en < 1 seconde	Automatisé
Phase 7	Un badge est attribué après une action et le XP est incrémenté	Manuel
Phase 8	L'app répond en < 200ms pour 95% des requêtes en production	Load test

12.2 Checklist Finale

- Toutes les migrations appliquées sans erreur
- Admin Django accessible et fonctionnel
- Authentification email + Google OAuth opérationnelle
- Cycle complet examen : création → composition → correction IA → bulletin PDF
- QCM : création questions → passage → résultat → feedback IA
- Paiement FedaPay sandbox → webhook → activation abonnement
- WebSockets : notifications temps réel opérationnelles
- Gamification : XP + badges fonctionnels
- HTTPS actif, security headers configurés
- Fichiers statiques servis par WhiteNoise
- Médias stockés sur Cloudinary
- Logs Django configurés (console + fichier)
- Variables d'environnement sécurisées (pas de secret en dur)
- Application déployée et accessible sur Render